

WEB-BASED VEHICLE AUCTION PLATFORM

**K.K.J. HAMID
R222226**

JULY - 2026



WEB-BASED VEHICLE AUCTION PLATFORM

HAMID K.K.J.
R222226

Supervisor: Dr. Subodha Hettiarachchi Gunawardena

JULY - 2026



**This dissertation is submitted in partial fulfilment of the requirement of the
Degree of Bachelor of Information Technology (External) of the
University of Colombo School of Computing**

1.1 Declaration

I certify that this dissertation does not incorporate, without acknowledgement, any material previously submitted for a degree or diploma in any University/Institute, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text.

Signature of Candidate:

Date:.....

Name of Candidate:

Countersigned by:

Signature of Supervisor(s)/Advisor(s):

Date:.....

Name(s) of Supervisor(s)/Advisor(s):

1.2 Abstract

TaproJapan Co. Ltd., a Japan-based vehicle exporter, managed its vehicle auctions through WhatsApp broadcasts and informal messaging. This process offered no audit trail, no buyer verification, and frequent disputes over who placed the highest bid, creating a scaling bottleneck as the client's international buyer base grew. This project designed and implemented a web-based vehicle auction platform to replace this manual workflow with a structured, transparent, and auditable digital system.

The system was built as a three-tier full-stack application: a React 19 single-page front end, a layered Node.js/Express REST API, and a PostgreSQL relational database. The backend separates routes, controllers, services, and repositories, with all business logic such as authentication, role-based access control, bid validation, and auction status transitions concentrated in the service layer. The database schema evolved through eight incremental migrations, moving from a basic four-table design to a normalized structure supporting auction-scoped bidding, hidden reserve prices, and a three-state buyer verification workflow.

Key functionality delivered includes JWT-based stateless authentication, an admin-only buyer verification workflow, a vehicle catalogue with multi-criteria search and filtering, a time-bound auction module with dual-layer status correction, minimum-increment bid validation, reserve-price privacy protection for non-admin users, and an admin dashboard for vehicle, auction, and user management with manual or automatic winner selection. Direct-to-Cloudinary image uploads keep the backend stateless with respect to binary data.

The system was verified through unit and integration tests covering authentication, vehicle CRUD, bidding rules, and auction lifecycle transitions, supplemented by manual end-to-end and User Acceptance Testing with the client. Testing surfaced and resolved several non-trivial defects, including an authentication flow that blocked all logins, a role-escalation vulnerability in registration, and an architectural flaw that scoped bids to vehicles rather than auctions. The resulting platform meets its core functional and non-functional requirements and gives the client a scalable, transparent replacement for its manual auction process, with payment integration and shipping logistics identified as natural extensions for future work.

1.3 Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Subodha Hettiarachchi Gunawardena, for his guidance, technical feedback, and continued support throughout the duration of this project. I am also grateful to the staff of the University of Colombo School of Computing for the resources and environment that made this work possible. My thanks also go to TaproJapan Co. Ltd. for their cooperation as the client of this project, for sharing their business process in detail, and for their feedback during user acceptance testing.

1.4 Table of Contents

When preparing a table of contents, use only two levels, list the main chapters as first-level headings (e.g., Introduction, Design, Implementation) and include important subheadings as second-level entries (e.g., 2.1 System Architecture, 2.2 Tools Used). Align page numbers neatly and ensure the numbering matches the actual document. To maintain consistency and accuracy, it is recommended to use the built-in table of contents and heading styles feature in your word processing application (such as Microsoft Word or Google Docs). This allows you to automatically generate and update the table of contents based on the heading levels used in your document. Ensure proper formatting, align page numbers neatly, and keep the layout clean and easy to navigate.

Table of Contents

Declaration	I
Abstract	II
Acknowledgements	III
Table of Contents	IV
List of Figures	V
List of Tables.....	VI
List of Acronyms.....	VII
Chapter 1 Introduction.....	1
Chapter 2 – Analysis.....	2
2.1 Analysis	3
Chapter 3 – Design.....	4
Chapter 4 – Implementation	6
Chapter 5 – Evaluation.....	8
Chapter 6 – Conclusion	9
References.....	10
Appendices.....	11
General Guidelines.....	12

1.5 List of Figures

All figures in the dissertation should be numbered and named using an appropriate caption. Numbering is done using chapter number and a sequence number (e.g. Figure 3.2 for second figure in Chapter 3). Figures in the appendices are numbered using the Appendix letter (e.g. Figure C.2 for second figure in Appendix C). List of figures consists of figure number, captions and page numbers. List can be generated using features of a word processing package. All figures used in the main chapters must be described in text prior to its use and must be referred to using its figure number. For example, in Section 3.5 of this document Figure 3.1 is referred to in text in the paragraph before the figure.

Figure 1.1: Sample Figure..... 1

1.6 List of Tables

All tables in the dissertation should be numbered and named using an appropriate caption. Numbering is done using chapter number and a sequence number (e.g. Table 3.2 for second table in Chapter 3). Tables in the appendices are numbered using the Appendix letter (e.g. Table C.2 for second table in Appendix C). List of tables consists of table number, captions and page numbers. List can be generated using features of a word processing package. All tables used in the main chapters must be described in text prior to its use and must be referred to using its table number. Example: In section 5.1 of this document Table 5.2 is referred to in text in the paragraph.

Table 1.1: Sample Table 2

1.6.1 List of Acronyms

API – Application Programming Interface

CRUD – Create, Read, Update, Delete

CDN – Content Delivery Network

CRC – Cyclic Redundancy Check (not used; placeholder removed)

ERD – Entity-Relationship Diagram

HTTP – Hypertext Transfer Protocol

JWT – JSON Web Token

ORM – Object-Relational Mapping

RBAC – Role-Based Access Control

REST – Representational State Transfer

SDLC – Software Development Life Cycle

SPA – Single Page Application

SQL – Structured Query Language

UAT – User Acceptance Testing

UI – User Interface

UUID – Universally Unique Identifier

Chapter 1 Introduction

1.1 Problem and Background

TaproJapan Co. Ltd. is a vehicle exporter based in Tokyo, Japan, operating as a small private business. The company purchases used vehicles at weekly Japanese auto auctions and resells them to international buyers in markets such as Sri Lanka, Kenya, and the Middle East. Prior to this project, the entire sales workflow of listing vehicles, collecting bids, and announcing winners was conducted manually through WhatsApp broadcast lists. Vehicle photographs and specifications were distributed informally with no consistent structure, buyers replied to message threads with their bids, and the client manually go through these replies to determine the highest bidder. Winner selection was communicated privately, which buyers who did not win perceived as opaque and potentially unfair.

This manual process produced several serious problems: there was no audit trail of bids, so disputes about who bid what and when could not be resolved authoritatively; there was no access control, meaning any user could submit a bid without any verification of identity or seriousness; vehicle information was scattered across different message threads with no central record; there was no historical database of past auctions, vehicles, or buyer activity; and the process did not scale. As the number of vehicles and buyers grew, the owners' ability to track bids manually became an operational bottleneck.

1.2 Motivation

As TaproJapan decided to expand their buyer base internationally, they expected the increasing volume of concurrent bidding conversations to make manual tracking more error prone. Missed bids, duplicate entries, and miscommunication regarding vehicle specifications were likely to become more frequent, directly affecting customer trust and satisfaction. Furthermore, there was no mechanism to restrict bidding to verified and financially committed buyers, making it difficult for the client to distinguish serious purchasers from casual enquiries. This could slow down decision-making and increase the risk of a winning bidder failing to follow through with the purchase.

The client's intention to grow the business from a small operation into a larger international export operation required a professional, automated, and scalable platform that could attract and manage a larger pool of buyers without proportionally increasing manual administrative work. From an academic standpoint, the project offered an opportunity to model a genuine small-business process which is an auction-based commerce with identity verification and time-bound bidding, using a modern full-stack web architecture, and to confront real engineering trade-offs such as data consistency under concurrent bidding, stateless authentication, and reserve-price confidentiality.

1.3 Aim and Objectives

The aim of this project is to design, implement, and deliver a web-based vehicle auction platform that digitizes and automates TaproJapan's auction workflow, replacing informal messaging with a transparent, auditable, and scalable system.

The specific objectives are:

1. To develop a web-based platform that streamlines the client's vehicle auction process and improves transparency for buyers, by replacing ad-hoc messaging with a structured catalogue, auction, and bidding system.
2. To ensure secure and trustworthy participation by implementing proper user registration, admin-driven verification, and controlled, role-based bidding access.
3. To support efficient auction management by providing administrators with tools for handling vehicle listings, auction scheduling, user verification, and winning-bidder selection.
4. To maintain accurate historical records of vehicles, auctions, and bids, and to ensure data integrity, reliability, and secure handling of all stored information.
5. To validate and refine the system through iterative testing and direct stakeholder feedback so the delivered system aligns with the client's actual business needs.

1.4 Scope of the Project

The scope of this project covers the full development lifecycle of a web-based vehicle auction platform intended for use by TaproJapan Co. Ltd. and its international buyers. The system provides three tiers of access. Unauthenticated guests can browse the public vehicle catalogue and auction listings, search and filter vehicles, and view the complete public bid history for any auction. Registered buyers can additionally place bids, but only once their account has been verified by an administrator; buyers can also view auctions they have won on their profile page. Administrators are created only through a one-time seed script rather than through public registration and have full CRUD control over vehicle listings and auctions, can verify or reject buyer accounts, can monitor bidding activity in real time, and can close auctions and select winning bids either automatically or manually.

The system targets a small-to-medium volume of concurrent users appropriate to TaproJapan's current and near-term business scale, and is designed to be deployed as a single-server full-stack web application (Node.js/Express backend, PostgreSQL database, statically-served React front end behind Nginx). The environment assumed is a standard web browser on desktop or mobile, with no native mobile application component.

Explicitly out of scope for this project are payment gateway integration, shipping and logistics management, and advanced analytics or business-intelligence reporting beyond basic auction-result summaries. These were identified jointly with the client as valuable but separable concerns that can be added as a second phase once the core auction workflow is in active use, and are revisited in the Future Work section of Chapter 6.

1.5 Outline of the Dissertation

Chapter 2 analyses the existing manual process in detail, reviews comparable systems, and derives the functional and non-functional requirements, including security as a first-class functional requirement, along with the justification for the chosen development methodology and technology stack.

Chapter 3 presents the system design, covering the three-tier architecture, the layered backend pattern, the entity-relationship model and its evolution, and the user interface design.

Chapter 4 describes the implementation: the development environment, module structure, coding conventions, key algorithms such as auction status correction and reserve-price stripping, and version control practices.

Chapter 5 reports on the evaluation of the system, covering unit and integration testing, end-to-end manual testing, and user acceptance testing with the client, including the defects discovered and resolved during this process.

Chapter 6 concludes the dissertation with a critical evaluation of the project's outcomes, a personal reflection on the learning experience, and recommendations for future work.

Chapter 2 – Analysis

2.1 Existing System and Problem Description.

Before this project, TaproJapan's auction workflow ran entirely through informal communication channels, summarized in Table 2.1.

Table 2.1 Existing Manual Auction Workflow

Phase	How it worked (old way)
Vehicle Listing	Photos taken manually; specifications recorded on paper; distributed via WhatsApp broadcast lists.
Bid Collection	Buyers reply to message threads. The client manually reads all replies and identifies the highest bid.
Winner Selection	The client privately messages the winning bidder. The process is opaque to other participants.

As shown in Figure 2.1, the process begins with the acquisition and listing of a vehicle, followed by the distribution of vehicle details to potential buyers through informal communication channels. Buyers submit bids via message threads, which must be manually monitored by the client. Once the bidding period is considered complete, the client identifies the highest bidder and proceeds with the transaction offline. This workflow presents several limitations, including the absence of a defined bidding period, lack of structured bid records, limited transparency, and the inability to verify bidder authenticity. These issues reduce operational efficiency and make the process difficult to scale as the number of vehicles and buyers increases.

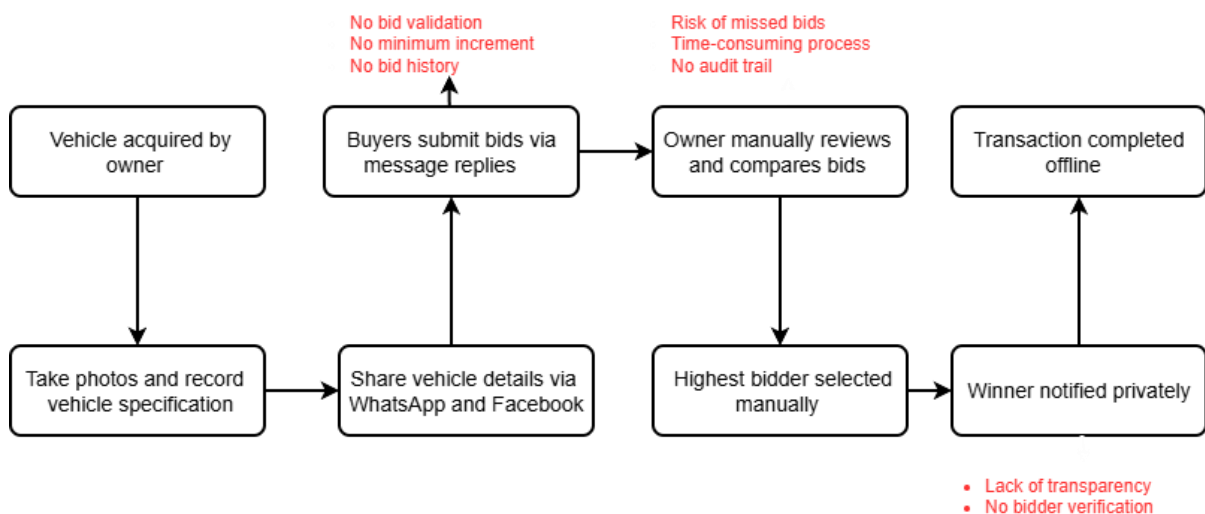


Figure 2.1 Existing Manual Vehicle Auction Workflow at TaproJapan

2.2 Review of Similar Systems

To inform the design of the platform, three categories of comparable system were reviewed: general consumer auction marketplaces, vertical vehicle-auction platforms, and verified-buyer marketplaces.

Consumer auction marketplaces (such as eBay-style time-bound auctions) provided the closest conceptual model for the core bidding mechanism. These systems typically employ fixed auction start and end times, competitive ascending bids, and publicly visible bid histories. This model was adopted for the proposed system because it promotes transparency and encourages competitive bidding among buyers.

Vertical vehicle-export auction platforms used by Japanese vehicle exporters typically incorporate vehicle-specific attributes such as chassis number, mileage, auction grade, and reserve prices. These platforms demonstrate the importance of structured vehicle data and auction-specific information management. Accordingly, the proposed system includes detailed vehicle records and support for reserve prices that remain hidden from non-admin users.

Verified-buyer marketplaces, commonly used in B2B and high-value asset trading environments, restrict participation to approved users rather than allowing open public bidding. This approach was particularly relevant to TaproJapan's requirements, as the client wanted to ensure that only verified and financially committed buyers could participate in auctions.

In addition to reviewing these categories of systems, several existing vehicle auction platforms were examined to identify practical design and workflow patterns. Platforms such as [Cars & Bids](#), [AutocomJapan](#), and [RamaDBK](#) provide reference implementations of web-based vehicle auction platforms. Analysis of these systems reveals common design patterns: paginated vehicle grids with rich filtering, countdown timers displayed to buyers, bid ladders showing historical offers, and admin dashboards for lot management. These patterns have been incorporated into the UI/UX design of this project.

Table 2.2 summarizes the comparison.

Table 2.2 Feature comparison of reviewed system categories

Feature	Consumer auctions	Vehicle-export platforms	Verified-buyer marketplaces	This project
Time-bound bidding	Yes	Yes	Sometimes	Yes
Hidden reserve price	Rare	Yes	Sometimes	Yes
Mandatory buyer verification	No	Sometimes	Yes	Yes
Public bid history	Yes	Sometimes	Rare	Yes
Minimum bid increment	Yes	Yes	Sometimes	Yes

The platform designed for this project therefore combines the open, transparent bid-history convention of consumer auctions with the verified-participation and hidden-reserve conventions of vehicle-export and B2B marketplaces, reflecting Tapro Japan's specific requirement for transparency toward genuine buyers without exposing commercially sensitive reserve information.

2.3 System Requirements Analysis

2.3.1 Functional Requirements

Functional requirements were elicited from direct discussion with the client and from the pain points identified in section 2.1. They are listed in Table 2.3 with priority and category.

Table 2.3 Functional requirements

ID	Requirement	Priority	Category
FR-01	User registration with email, password, and contact information	High	User Management
FR-02	Admin verification or rejection of registered user accounts	High	User Management
FR-03	Secure login with JWT-based authentication	High	User Management
FR-04	Role-based access control: public, verified buyer, admin	High	User Management
FR-05	Admin CRUD operations for vehicle listings (images, specs, auction date)	High	Vehicle Management
FR-06	Batch upload of weekly auction vehicles	Medium	Vehicle Management
FR-07	Public vehicle browsing with search and filter (make, model, year, price)	High	Vehicle Management
FR-08	Creation of auctions with configurable start/end datetime	High	Auction Management
FR-09	Automatic auction status transitions	High	Auction Management
FR-10	Verified users may place bids on active auctions	High	Bidding System
FR-11	Bid validation: minimum increment enforcement, auction timing check	High	Bidding System
FR-12	Real-time display of current highest bid to buyers	High	Bidding System
FR-13	Prevention of bidding after auction closure	High	Bidding System
FR-14	Complete bid history per auction, accessible to admin	High	Bidding System
FR-15	Admin dashboard: pending verifications, active auctions, bid summaries	High	Admin
FR-16	Admin selection of winning bidder per auction	High	Admin
FR-17	Email/in-app notifications for verification status and outbid events	Medium	Notifications
FR-18	Basic reporting: auction results, top buyers, bid counts	Medium	Reporting

2.3.2 Security as a Functional Requirement

Security was treated as a core functional requirement rather than a non-functional afterthought, since the system's central value proposition is trustworthy, auditable bidding. Four security controls were built directly into the system's functional behaviour. Access control is enforced through JWT-based role checks at the middleware layer (`authRequired` and `requireRole`), so every protected endpoint explicitly declares which roles may call it. Admin account creation is excluded entirely from the public API surface ; administrators can only be created by a database seed script run by a developer with direct database access, preventing any client-side request from escalating itself to admin. Input validation is enforced through Joi schemas at the route layer for every mutating endpoint, rejecting malformed or unexpected fields before they reach business logic. Data protection is enforced through bcrypt password hashing (cost factor ≥ 10), parameterised SQL queries on every repository call to prevent injection, and reserve-price values being stripped from API responses at the service layer before they ever reach a non-admin client, rather than being hidden only in the front end. Treating these as functional requirements meant each had an explicit, testable behaviour rather than being an implicit assumption.

2.3.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints of the system beyond its core functionality. These requirements ensure that the platform performs reliably, remains secure, and provides a satisfactory user experience. The non-functional requirements identified for the proposed system are summarised in Table 2.4.

Table 2.4 Non-functional requirements

Category	Requirement
Performance	API responses should complete in under 500 ms under normal load; the system should support at least 100 concurrent users.
Reliability	The system should maintain 99% uptime during active auction periods; database transactions must guarantee consistency for concurrent bid submissions.
Usability	The interface must be responsive across desktop and mobile screen sizes; returning buyers should require no training to use core features.
Scalability	The backend must be stateless (no server-side session store) so it can be deployed behind a load balancer if buyer volume grows; the database must use connection pooling.
Maintainability	The backend must enforce strict separation between routes, controllers, services, and repositories so that a change to one concern (e.g., a new database column) does not require changes to unrelated layers.
Security	See Section 2.3.2

2.4 Justification of Development Approach and Technologies

2.4.1 Chosen SDLC Model and Rationale

An iterative Agile-inspired development approach was adopted for this project. Although the work followed a broadly sequential progression from requirements analysis to deployment, development was carried out through multiple implementation and testing cycles, allowing features to be refined based on client feedback and system testing results.

The project commenced with requirements analysis and system design activities, including the preparation of the ER diagram, class diagram, sequence diagrams, and Software Architecture Design (SAD) documentation. Following completion of the design phase, backend services and database functionality were implemented, including user management, authentication, vehicle management, auction handling, and bid processing features.

Once the core backend functionality was operational, frontend development was carried out incrementally, integrating completed API endpoints into the user interface. The auction and bidding module received particular attention due to its importance to the overall system and the need to ensure correct bid validation, auction timing, and winner selection logic.

Testing was performed continuously throughout development, with dedicated efforts focused on unit testing, integration testing, security improvements, bug fixing, and user interface refinements. Several enhancements requested by the client were incorporated during later iterations.

This methodology was selected because the project involved a real client whose requirements evolved during development. The ability to implement, test, and refine features incrementally reduced project risk, enabled regular validation of system behaviour, and ensured that the final product aligned closely with the client's operational requirements. The major development phases are summarised in Table 2.5.

Table 2.5 Major Development Phases

Phase	Deliverable
Phase 1	Requirements Analysis and Problem Definition
Phase 2	System Analysis and Design (ERD, UML Diagrams, SAD)
Phase 3	Backend API and Database Development
Phase 4	Frontend Development and System Integration
Phase 5	Auction and Bidding Module Implementation
Phase 6	Testing, Security Improvements, and Bug Fixing
Phase 7	Documentation, Client Feedback Integration, and Final Deployment

2.4.2 Technology stack and justification

Table 2.6 Technology stack and justification

Layer	Technology	Justification
Frontend	React 19 + Vite + Tailwind CSS v4	A component-based SPA model suits a UI with many dynamic, frequently-updating regions (live bid amounts, countdown timers, status badges). Vite was chosen for its fast hot-module-reload during development. React was preferred over Vue for its larger ecosystem and longer-term maintainability for a single-developer project.
Backend	Node.js 20 + Express 4	Non-blocking I/O is well suited to handling many concurrent bid submissions without thread-per-request overhead. Using JavaScript across both frontend and backend reduced context-switching for a solo developer. Express's minimalism allowed the strict custom layered architecture (Section 3.2) to be enforced explicitly rather than fighting a more opinionated framework. Django/Python was considered and rejected due to added deployment complexity without a corresponding benefit for this project's scale.
Database	PostgreSQL 16	ACID-compliant transactions are essential to guarantee bid integrity under concurrent submissions. PostgreSQL's native array type (TEXT[]) was used directly for vehicle image URLs, avoiding a separate join table. Its NUMERIC type guarantees exact decimal arithmetic for currency, and its UNIQUE constraint semantics (treating each NULL as distinct) suited the optional chassis_number field. MySQL was considered and rejected due to weaker SERIALIZABLE isolation guarantees and the absence of a native array type.
Authentication	JWT (jsonwebtoken)	A stateless token-based scheme avoids a server-side session store, simplifying horizontal scalability, and allows role and verification claims to travel with the request without an extra database lookup on every call.
Image storage	Cloudinary (free tier)	Unsigned browser-to-CDN upload keeps the backend stateless with respect to binary data and avoids the performance and backup complications of storing images in PostgreSQL or on the application server's local disk.
Validation	Joi	Declarative schema validation with built-in support for stripping unknown fields and collecting all validation errors in a single response, rather than failing on the first error.
Scheduling	node-cron	A lightweight in-process scheduler was sufficient at this project's scale; an external job queue was judged to be unnecessary complexity.

Chapter 3 – Design

3.1 Introduction to System Design

This chapter translates the requirements defined in Chapter 2 into a concrete architectural and data design. The guiding design principle throughout is separation of concerns: each layer of the backend, and each major frontend module, has exactly one responsibility, which directly supports the maintainability and testability non-functional requirements (Table 2.4).

3.2 System Architecture Overview

The system follows a three-tier architecture, illustrated conceptually in Figure 3.1 and detailed in Table 3.1.

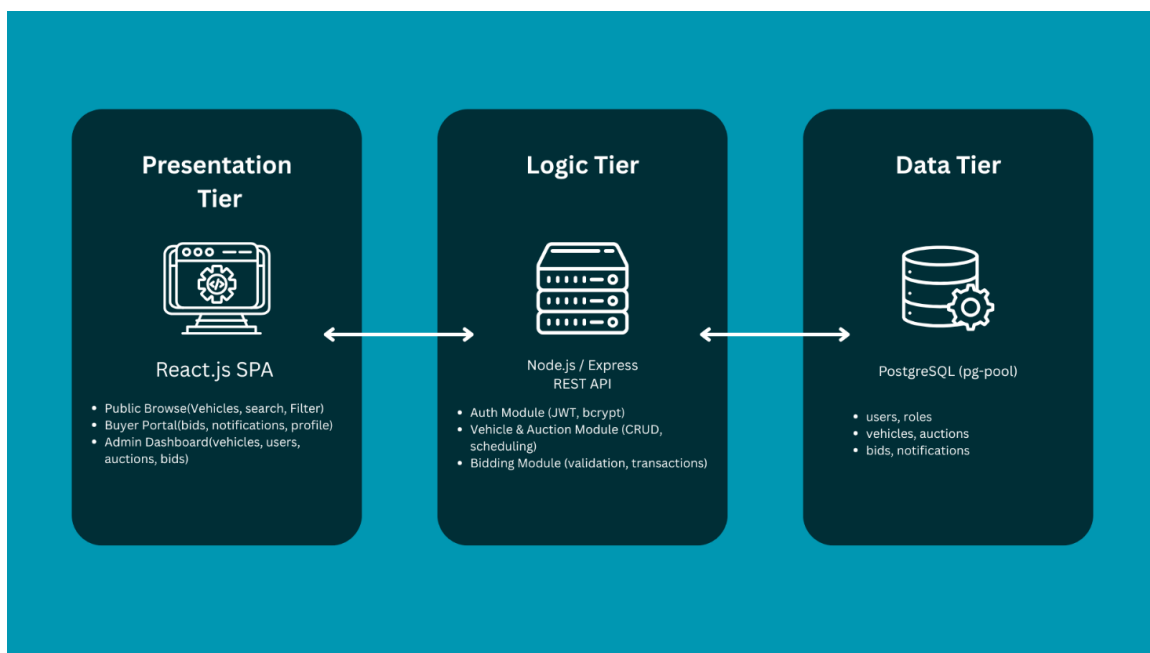


Figure 3.1 System High Level Architecture

Table 3.1 Three-tier architecture responsibilities

Tier	Technology	Responsibility
Presentation	React 19 + Vite + Tailwind CSS v4	Renders the UI, manages local component state, and communicates with the backend exclusively over HTTP via a single Axios instance. Contains no business logic.
Logic	Node.js + Express	Exposes a REST API; enforces authentication, authorisation, and all business rules.
Data	PostgreSQL	Relational storage with ACID transactions for bid integrity, accessed through a pooled connection (pg.Pool).

Within the application tier, the backend follows a strict four-layer pattern, shown as a request trace in Figure 3.2

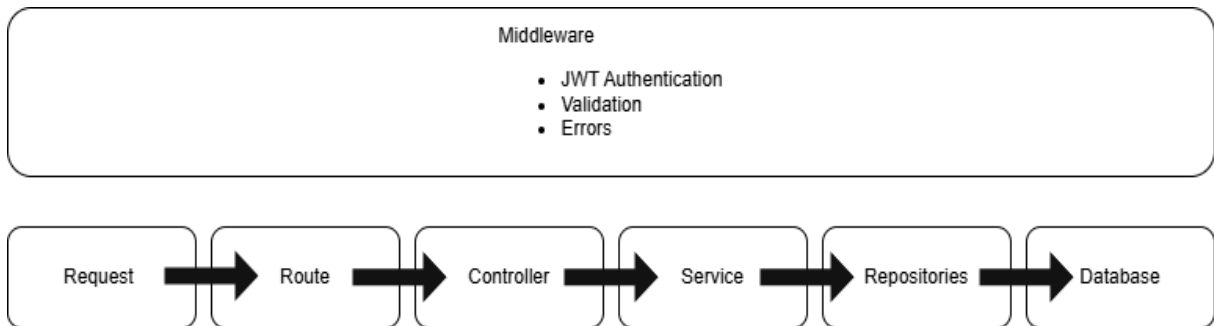


Figure 3.2 Backend Request Processing Architecture

Table 3.2 defines the responsibility of each layer precisely, since this separation is the central architectural decision of the backend.

Table 3.2 Backend layer responsibilities

Layer	Location	Responsibility
Routes	src/routes/	URL patterns and middleware chains only. Zero business logic.
Controllers	src/controllers/	Thin glue: call exactly one service method, then send the response or forward the error via <code>next(err)</code> .
Services	src/services/	All business logic, authorisation decisions, and cross-entity rules.
Repositories	src/repositories/	SQL queries only, using parameterised statements. No business logic.
Middleware	src/middleware/	JWT verification, role enforcement, Joi validation, and global error handling.

This separation was chosen, rather than a more conventional MVC structure with logic embedded in controllers, for three reasons. First, testability: services can be unit-tested without spinning up the HTTP layer, and repositories can be mocked independently. Second, security: because all authorisation logic lives in the service layer rather than the controller, it is enforced consistently even if a service method is called from a different entry point (for example, from an automated test or, in future, from a background job), rather than relying on every controller to remember to re-check permissions. Third, change isolation: a schema change such as adding a new vehicle column requires editing only the corresponding repository function; no other layer needs to know about column names.

3.3 Component and Module Design

3.3.1 Backend Modules

The backend is organised into one repository, one service, one controller, and one route file per primary resource: users, vehicles, auctions, and bids. Each repository exposes narrow, single-purpose functions (for example, `findByEmail`, `findById`, `create`, `findAll`, `setStatus` for the user repository). The auction repository's `findAll` and `findById` queries join the vehicles table to surface denormalised display fields (`vehicle_make`, `vehicle_model`, `vehicle_year`, `vehicle_images`) and use a correlated subquery, (`SELECT MAX(amount) FROM bids WHERE auction_id = a.id`) AS `highest_bid`, to compute the current highest bid live rather than maintaining a separately stored column that could fall out of sync with the bids table. The vehicle repository's `findAll` function supports dynamic filtering across up to nine optional conditions (status, make, model, year range, price range, free-text search), building a parameterised WHERE clause and running the query twice: once with `COUNT(*)` for pagination metadata and once with `LIMIT/OFFSET` for the page of results.

3.3.2 Frontend Modules

The frontend follows a similar single-responsibility convention. One Axios instance (`src/api/client.js`) carries a request interceptor that attaches the JWT Bearer token from `localStorage` to every outgoing request; one thin wrapper file per resource (`auth.js`, `users.js`, `vehicles.js`, `auctions.js`, `bids.js`, `cloudinary.js`) exposes typed functions over that instance, so no component is permitted to call Axios directly. Pages own their own data-fetching and loading/error state using `useState` and `useEffect`. Cross-cutting concerns that do not fit a parent/child component relationship, specifically, notifying the Navbar's pending-user-count badge when `AdminPage` approves or rejects a buyer are handled with a custom DOM event (`window.dispatchEvent(new CustomEvent('user-status-changed'))`) rather than prop drilling or a global state library, which was judged proportionate given the small number of cross-component signals required.

3.4 Workflow and Behavioural Modelling

3.4.1 End-to-End Sale Workflow

The vehicle auction process involves interactions between administrators, buyers, and the system itself. Figure 3.3 illustrates the complete lifecycle of an auction, beginning with vehicle listing creation and ending with winner notification. The workflow captures both user-driven activities and automated system operations such as auction activation and closure.

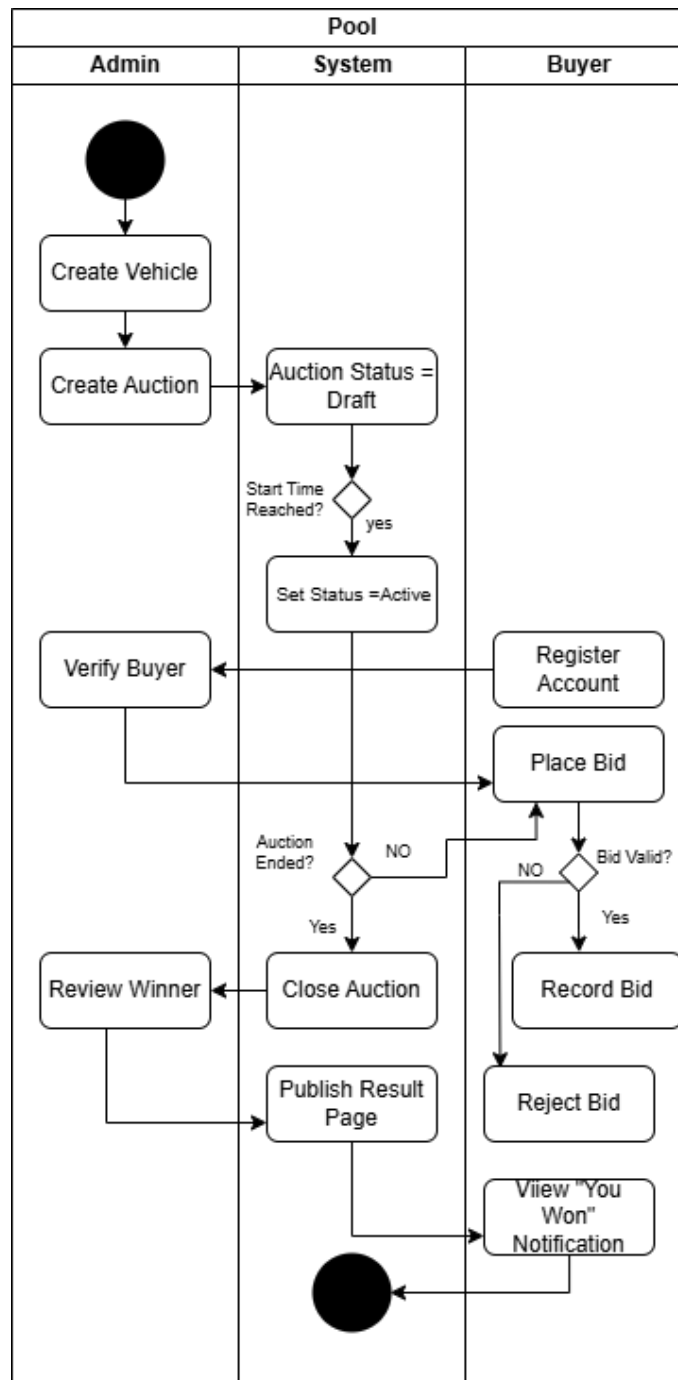


Figure 3.3 Activity Diagram for the End-to-End Vehicle Auction Workflow

As shown in Figure 3.3, the administrator first creates a vehicle listing and configures an associated auction by defining the auction period, reserve price, and minimum bid increment. The auction initially remains in the draft state until the configured start time is reached, at which point the system automatically activates the auction. Buyers must register and obtain administrator approval before they are permitted to participate in bidding.

During the active auction period, verified buyers may place bids that satisfy the validation rules enforced by the system. Bids that do not exceed the current highest bid by the required minimum increment are rejected. Once the auction end time is reached, the system closes the auction and identifies the highest bid. The administrator may then review the bid history and confirm or override the selected winner if necessary.

Following winner confirmation, the auction result becomes publicly available through the winner page, and the successful bidder is notified through the application

3.4.2 Auction Status Transition Logic

Auction status is modelled as a three-state machine (draft, active, ended) driven by two independent mechanisms working together, since relying on either alone was found insufficient (see Bug #7 in Chapter 5). A cron job (node-cron, running every minute) executes two bulk UPDATE statements against the whole auctions table, transitioning draft auctions whose starts_at has passed to active, and active auctions whose ends_at has passed to ended. Independently, a pure function, effectiveStatus(auction), is evaluated on every read of an auction and recomputes the status from the stored timestamps; if it disagrees with the persisted status, the service immediately writes the corrected value back to the database. This guarantees that the status displayed to any client is always accurate at the moment of the request, while the cron job ensures the database itself self-corrects even when no request is being made.

3.5 Data Modelling

3.5.1 Entity-Relationship Overview

The initial database design was intentionally minimal and focused on validating the core auction workflow. As implementation progressed and additional requirements emerged, several limitations of the original schema became apparent. The original design stored bids directly against vehicles, which proved insufficient because bidding activity is associated with a specific auction event rather than the vehicle itself. This was corrected by introducing the auction_id foreign key on the Bid entity. Similarly, the original boolean is_verified attribute could only represent two states and was unable to distinguish between newly registered users awaiting approval and users who had been explicitly rejected. This limitation led to the introduction of the verification_status attribute with pending, verified, and rejected states.

Additional schema changes were driven by functional requirements identified during development. Vehicle-specific attributes such as chassis number, mileage, grade, and image collections were introduced to support detailed vehicle listings. The Auction entity was extended with min_increment and reserve_price attributes to support bid validation and hidden reserve-price functionality. Finally, the winning_bid_id relationship was introduced to explicitly record the selected winner of an auction, improving traceability and simplifying result publication. These changes transformed the original proof-of-concept schema into a production-oriented data model that more accurately reflected the business processes of Tapro Japan.

The final data model comprises four core entities: Users, Vehicles, Auctions, and Bids. An administrator user may own multiple vehicle listings through the seller_id foreign key. Each vehicle may be associated with an auction through the vehicle_id foreign key, while each auction may contain multiple bids linked through auction_id. Every bid belongs to exactly one user via the user_id foreign key. To support winner selection, the Auction entity optionally references a winning bid through the nullable winning_bid_id foreign key. Referential integrity is maintained through foreign key constraints, with cascading deletes applied where appropriate.

3.5.2 Schema Evolution

The schema was not designed in a single pass; it evolved through eight incremental, numbered migrations, each solving a specific problem identified during analysis or development, summarised in Table 3.3. This incremental approach follows the standard professional practice of never editing a migration after it has been applied to a real database every change is captured as a new file, preserving a complete, reproducible history of how the schema reached its final state.

Table 3.3 Migration history

Migration	Change	Rationale
001	Initial schema: users, vehicles, auctions, bids	Establishes the four core entities and basic auction workflow
002	Replace bids.vehicle_id with bids.auction_id	Bids belong to a specific auction event rather than directly to a vehicle
003	Add winning_bid_id to auctions	Records the winning bid selected for an auction
004	Add chassis_number, mileage, grade, images[] to vehicles	Supports detailed vehicle specifications and image management
005	Replace is_verified with verification_status	Supports pending, verified, and rejected account states
006	Add min_increment to auctions	Enforces minimum bid increment validation
007	Add reserve_price to auctions	Supports hidden reserve-price functionality
008	Add UNIQUE constraint on chassis_number	Prevents duplicate vehicle registrations
009	Add must_change_password to users	Supports mandatory password changes for temporary or seeded accounts

3.5.3 Key Data Modelling Decisions

NUMERIC over FLOAT for currency. All monetary fields (starting_price, amount, min_increment, reserve_price) use NUMERIC(12,2) rather than a floating-point type. Floating-point binary representation cannot exactly represent all decimal fractions, which is unacceptable for financial values where every cent matters; NUMERIC stores an exact decimal value.

Computed highest_bid rather than a stored column. An earlier design considered storing current_highest_bid as a column on the auction, updated whenever a bid was inserted. This was rejected because it introduces a denormalization risk: if the bid insert succeeds but a subsequent update to the cached column fails (or is skipped due to a bug), the displayed highest bid silently diverges from the true value in the bids table. Computing it live via a correlated subquery removes this entire class of bug at a modest, acceptable query cost.

Enumerated verification_status over a boolean. The original design used a boolean is_verified flag. This could not represent the required three-state workflow (pending, verified, rejected) specifically, it could not distinguish a buyer who has registered but not yet been reviewed from one who has been explicitly rejected. Migration 005 replaced the boolean with a VARCHAR column constrained by a CHECK clause to these three values.

Inline CHECK constraints over lookup tables. An earlier ERD modelled role and auction/verification status as separate lookup tables with their own primary keys, following conventional normalisation practice. This was deliberately simplified to inline CHECK constraints, because these are small, fixed sets of values that will never be added to or removed from at runtime; a lookup table would only add unnecessary JOIN overhead to the

authentication path and every auction status query, with no corresponding gain in data integrity.

Eliminating the seller role. The original requirements analysis considered a distinct "seller" role separate from "admin". This was removed during analysis because, in Tapro Japan's actual business model, the owner is always the seller it is a one-to-many relationship between one business and many buyers, not a marketplace with multiple independent sellers. Retaining a separate seller role would have added permission-checking complexity with no corresponding business value. The `seller_id` column was kept on the vehicles table (it always refers to the admin who created the listing, useful for audit purposes) but the role itself was removed from registration, route guards, and service logic.

3.6 User Interface Design

The interface was designed around three guiding principles: transparency (any visitor, logged in or not, can see the full public bid history and current highest bid for any auction), minimal friction for returning buyers (no training should be required, status is communicated through colour-coded badges and plain-language banners rather than jargon), and responsiveness across desktop and mobile, implemented with Tailwind CSS v4's utility classes rather than custom per-component CSS.

Key screens designed include: a public vehicle catalogue page with URL-synchronised filters (so that filtering, pagination, and even back/forward browser navigation all preserve and restore the correct query state) and skeleton-loading placeholders while data is in flight; an auction detail page, the most visually complex screen combining a lightbox image gallery, a live countdown timer, a colour-coded reserve-price indicator (no reserve / not met / met, with the actual amount visible only to admins), a bid submission form shown only to verified, authenticated, non-admin users while the auction is active, and (for admins) a "Close Auction" control and a bid ladder for manual winner selection; and an admin dashboard organised into three tabs (Vehicles, Auctions, Users), with the Users tab providing per-user verify/reject actions that immediately update a pending-count badge in the navigation bar via the custom event mechanism described in Section 3.3.2.

Chapter 4 – Implementation

4.1 Development and Deployment Environment

Development was carried out on Microsoft Windows using Visual Studio Code as the primary development environment. The backend was implemented using Node.js 20 LTS, PostgreSQL 16, and ECMAScript Modules, with dependencies including Express, pg, bcrypt, jsonwebtoken, Joi, Morgan, node-cron, and dotenv. Nodemon was used during development to automatically restart the server when source files changed. The frontend was developed using React and Vite, with React Router for client-side routing, Axios for API communication, and Tailwind CSS for styling. Git was used throughout the project for version control and change tracking.

Environment-specific configuration such as database connection strings, JWT secrets, and Cloudinary credentials was managed through environment variables stored in .env files and excluded from version control. This approach ensured that sensitive information was not exposed within the source repository.

The backend application and PostgreSQL database were deployed using Railway, which provides managed application hosting, automated builds, and database services. The frontend application was deployed separately on Vercel, which provides static hosting and content delivery network (CDN) support for React applications. This deployment architecture simplified infrastructure management by eliminating the need for manual server provisioning, reverse proxy configuration, and process management while still providing reliable application hosting and automated deployment from the Git repository.

4.2 Development Practices and Coding Standards

The codebase is organised according to the four-layer architecture described in Section 3.2, with each resource implemented through a dedicated route, controller, service, and repository. A consistent naming convention is followed throughout the project: repository, service, and controller files are named according to the resource they manage (e.g.,

`vehicleRepository.js`, `auctionService.js`, and `bidController.js`), while route files are named after the corresponding resource in plural form (e.g., `vehicles.js` and `auctions.js`).

Environment variables are accessed through a single configuration module, `src/config/index.js`, rather than being read directly via `process.env` throughout the application. This centralised approach improves maintainability and simplifies code review by ensuring that configuration values and secrets are managed consistently. The application entry points are separated into two files: `app.js`, which constructs the Express application and registers middleware and routes, and `server.js`, which starts the HTTP server and cron scheduler. This separation allows automated tests to import `app.js` directly without unintentionally starting background services or opening network ports.

Error handling follows a uniform convention across the service layer. When a business-rule violation is detected, the service throws a standard `Error` object with an attached `status` property (for example, `err.status = 404`). A single global error-handling middleware then reads this property and generates the corresponding HTTP response. This approach ensures that the meaning of each error is defined once within the layer that detects it, rather than being repeatedly interpreted by individual controllers, resulting in more consistent and maintainable error handling throughout the application.

4.3 Critical Code Segments

4.3.1 Authentication Middleware

4.3.2 4.3.1 Authentication Middleware

Two exported middleware functions guard protected routes. `authRequired` reads the `Authorization: Bearer <token>` header, calls `jwt.verify()`, and, on success, sets `req.user` to the decoded payload containing the user's database ID (`sub`), role, email, and verification status. On failure, it returns an HTTP 401 (Unauthorized) response. `requireRole(roles)`, which must execute after `authRequired`, checks whether `req.user.role` is included in the allowed roles array and returns an HTTP 403 (Forbidden) response otherwise. These middleware functions are composed declaratively within route definitions. For example:

```
router.post(
  '/',
  authRequired,
  requireRole(['admin']),
  validate(vehicleSchema),
  createVehicleController
);
```

This approach makes the access-control rules for each endpoint directly visible within the route definition.

4.3.3 4.3.2 Auction Status Correction

The `effectiveStatus()` function in `auctionService.js` is the core algorithm underpinning the dual-layer status-transition mechanism described in Section 3.4.2:

```
export function effectiveStatus(auction) {
  const now = new Date();

  if (
    auction.status === 'draft' &&
    auction.starts_at &&
    new Date(auction.starts_at) <= now
  ) {
    return 'active';
  }

  if (
    auction.status === 'active' &&
    auction.ends_at &&
    new Date(auction.ends_at) <= now
  ) {
    return 'ended';
  }

  return auction.status;
}
```

This function is pure and has no side effects, making it straightforward to unit test against a range of timestamp combinations. The calling service compares the computed status against the persisted database value and issues a write-back update only when the two values differ, thereby minimising unnecessary database writes.

4.3.4 4.3.3 Reserve Price Privacy

Reserve-price confidentiality (FR-15) is enforced at the service layer rather than relying on the frontend to hide sensitive information:

```
function stripReserve(auction, user) {
  const reserveMet =
    auction.reserve_price == null
      ? null
      : (
        auction.highest_bid != null
          ? Number(auction.highest_bid) >= Number(auction.reserve_price)
          : false
      );

  if (user?.role === 'admin') {
    return { ...auction, reserve_met: reserveMet };
  }

  const { reserve_price, ...rest } = auction;
  return { ...rest, reserve_met: reserveMet };
}
```

Administrators receive the complete auction object, including the reserve price. All other users receive the same object with the `reserve_price` field removed, while still receiving a `reserve_met` indicator (or `null` if no reserve price exists). This allows buyers to view meaningful reserve-status information without exposing the underlying reserve amount.

4.3.5 4.3.4 Bid Placement Validation

`bidService.placeBid()` performs an ordered sequence of validation checks, each throwing a descriptive error with an appropriate `.status` value on failure. First, the user's verification status is checked using information stored in the JWT payload, avoiding an additional database query. Next, the existence of the referenced auction is verified. The service then evaluates `effectiveStatus(auction)` to ensure that the auction is currently active, distinguishing between auctions that have not yet started and those that have already ended so that accurate error messages can be returned. Finally, the submitted bid amount is validated to ensure that it exceeds the current highest bid by at least the configured `min_increment` value, or simply exceeds the highest bid when `min_increment` is zero.

This validation order—verification, existence, timing, and amount—was chosen so that the most common and informative failure condition, an unverified user attempting to bid, is reported before further validation logic is executed.

4.3.6 4.3.5 Partial Update Merging

`vehicleService.updateVehicle()` (and the equivalent auction-update implementation) first retrieves the existing database record and merges it with the incoming payload using the nullish-coalescing operator rather than passing the raw request body directly to an `UPDATE` statement:

```
const merged = {
  title: payload.title ?? existing.title,
  make: payload.make ?? existing.make,
  // remaining fields follow the same pattern
};
```

This approach uses the value supplied in the request only when it has been explicitly provided, falling back to the existing database value otherwise. The pattern supports partial updates through HTTP PUT/PATCH endpoints and was introduced specifically to resolve the defect described in Chapter 5 (Bug #6), where omitted fields were being unintentionally overwritten with `NULL`.

4.4 Version Control Practices

Git was used throughout the project to manage source code, track changes, and support incremental development. The repository was organised into two primary directories, `Backend/` and `Frontend/`, each containing its own dependencies and configuration files. A root-level `.gitignore` file excluded generated dependencies (`node_modules/`), environment configuration files (`.env`), build artefacts (`dist/`), and operating-system-specific files from version control.

Commits were created regularly throughout development, with commit messages describing the functionality, bug fixes, testing activities, security improvements, and client-requested changes being implemented. The commit history therefore provides a chronological record of the system's evolution from requirements analysis and system design through implementation, testing, deployment, and maintenance activities.

Database schema changes were managed through numbered migration files. Once a migration had been applied to a database instance, it was never modified directly. Instead, all subsequent schema changes were introduced through new migration files, preserving a complete and reproducible history of database evolution as described in Section 3.5.2.

4.5 Testing Tooling Notes

Several environment-specific considerations were encountered during testing and were resolved to ensure reproducibility across development environments. The backend application was implemented using ECMAScript Modules (`"type": "module"`), which required additional configuration when running tests with Jest. Because Jest's default execution model is based on CommonJS modules, tests were executed using Node.js's `--experimental-vm-modules` flag to enable compatibility with ECMAScript Modules.

A second issue was encountered on Windows systems when invoking Jest through the executable located in `node_modules/.bin`. The generated launcher script is intended for POSIX-compatible shells and therefore produced execution errors when invoked directly in certain Windows environments. This issue was resolved by executing Jest through its JavaScript entry point located in `node_modules/jest/bin/jest.js`, providing a platform-independent solution.

The final test command used during development was:

```
node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand --forceExit
```

This configuration ensured consistent execution of automated tests across the development environment while maintaining compatibility with the project's ECMAScript Module architecture.

Chapter 5 – Evaluation

5.1 Testing Strategy Overview

The system was verified at four levels: unit testing of pure business-logic functions, integration testing of complete API request/response cycles, manual system testing via a structured end-to-end checklist, and User Acceptance Testing with the client. A dedicated test database (`vehicle_auction_test`), schema-identical to the development database but populated independently, was used for all automated tests, with its own `.env.test` configuration, so that test runs could never affect or be affected by development data.

5.2 Unit and Integration Testing

Automated tests were written using Jest, organised one test file per resource, as summarised in

Table 5.1 Automated test coverage by file

Test file	Coverage
<code>auth.test.js</code>	Buyer registration, rejection of attempted admin self-registration, duplicate-email handling, password validation, login and token issuance, and 401 responses for missing/invalid credentials
<code>vehicles.test.js</code>	Public listing, and create/update/delete operations verified against 401/403/200/204 outcomes per role, plus the filtering and pagination query suite
<code>users.test.js</code>	Verify/reject workflow, the corresponding 401/403/404/409 error cases, confirmation that a freshly issued token correctly reflects <code>isVerified: true</code> after verification, and the <code>/users/me</code> endpoint
<code>bids.test.js</code>	Rejection of bids from unverified buyers, successful bids from verified buyers, the outbid/minimum-increment enforcement rule, and rejection of bids placed by an admin account
<code>auctions.test.js</code>	Auction CRUD, the close-auction action, manual winner selection, the public winner endpoint, and the automatic status-transition logic

Because services were designed to be callable independently of the HTTP layer (Section 3.2), pure functions such as `effectiveStatus()` could be tested directly against a range of timestamp inputs without needing to mock an HTTP request at all, which kept these particular tests fast and deterministic.

Chapter 6 – Conclusion

6.1 Critical Evaluation

The delivered system meets its core functional requirements: secure registration and authentication, admin-controlled buyer verification, full vehicle CRUD with search and filtering, time-bound auctions with automatically and reliably correct status, minimum-increment bid validation, public bid transparency, hidden reserve pricing with a buyer-visible indicator, and both automatic and manual winner selection. The non-functional requirements around security (parameterised queries, bcrypt hashing, role-based access control enforced server-side) and maintainability (the strict layered backend architecture) were also achieved and were directly responsible for making the defects described in Chapter 5 findable and fixable in isolation — for example, the bid-scoping defect (Bug #5) required changes only to the bids repository and a migration, not to any controller or route.

Some trade-offs were made deliberately. The JWT-based authentication scheme accepts a known staleness window: a buyer's verification status embedded in their token does not update until they log in again, even if an admin verifies them in the interim. This was judged an acceptable cost for the simplicity and statelessness gained by avoiding a server-side session store, and is mitigated by having the profile page always read live data from `/api/users/me` rather than trusting the token. Similarly, the dual-layer auction status correction (cron plus on-read) accepts up to a one-minute window where the database's stored status could theoretically lag reality if no request happens to arrive in that window; in practice, the on-read correction means no client-visible response is ever stale, only the raw database row briefly is.

The system's main current limitation, by deliberate scope decision rather than oversight, is the absence of payment and shipping-logistics integration, meaning the platform manages the auction and winner-selection process but the actual transaction and delivery still proceed offline, as documented in Section 1.4.

6.2 Personal Reflection

This project required applying software development principles across the full stack rather than in isolation, and the most valuable learning came specifically from the defect-discovery process documented in Chapter 5. Several of the defects found — particularly the bid-scoping architectural flaw and the login-blocking authentication bug — were not surface-level mistakes but consequences of an initial design that had not been thought through against the actual business requirement closely enough; finding and correctly diagnosing them required systematically tracing requirements through to implementation rather than only testing happy-path behaviour. Working independently, without a team to catch design flaws through review, reinforced the value of writing the layered architecture early, since it made these defects locatable and fixable in single, contained files rather than requiring wide, risky refactors. The experience also developed practical skills in incremental database schema management through migrations, in writing tests that catch real defects (Bug #11 was found by a test assertion, not by manual inspection), and in communicating technical trade-offs (such as the JWT staleness window) to a non-technical client in a way that let them make an informed decision rather than treating it as a hidden flaw.

6.3 Future Work

Several extensions were identified jointly with the client as valuable next steps beyond the current scope. Payment integration, allowing a winning buyer to place a deposit or full payment directly through the platform, was the most frequently requested addition during client discussions, though it was correctly excluded from this project's scope given the additional compliance and security surface area it would introduce. Shipping and logistics tracking, allowing the client to record and the buyer to view the export and delivery status of a won vehicle, would extend the system's usefulness beyond the point of sale. Real-time bid updates via WebSockets (Socket.io) could replace the current 15-second polling interval on the auction detail page, reducing latency between a bid being placed and other viewers seeing it, at the cost of additional infrastructure complexity that was judged unnecessary at the project's current scale. Finally, basic analytics — auction-level summaries of bid counts, participation rates, and price trends over time — would help the client identify which categories of vehicle attract the strongest buyer interest, supporting better purchasing decisions at the Japanese auctions that supply the platform.

References

It is very important to acknowledge any of the work of others that the candidate used or adapted in the project, or that provided the essential background or context to the dissertation. Please note that IEEE is the recommended referencing and citation style for your dissertation. It is also recommended that the candidate use a reference management tool such as Mendeley or Zotero

Following examples show how the referencing should be done.

7.1.1 Example 1:

Systems analysis and design techniques are considered essential for developing client/server and web-centric applications [6, 7].

7.1.2 Example 2:

Software testing [8] is an iterative process.

7.1.3 Example 3:

“Plagiarism is an act of fraud. It involves both stealing someone else's work and lying about it afterward” [9].

In the **References** section, each citation should be listed in the relevant format (Refer to an IEEE referencing and citation style guide). For example, the reference section entries for the above three examples would be;

[6] J.L. Whitten and L.D. Bentley, *Systems Analysis and Design Methods*, 7th ed. Tata McGraw-Hill, 2007.

[7] UCSC, *The Virtual Learning Environment for the BIT Students*, 2006. [Online] Available: <http://vle.bit.lk> , [Accessed: 30 Oct, 2013]

[8] I. Sommerville, *Software Engineering*, 8th ed. Addison-Wesley, 2006.

[9] Plagiarism.org - Best Practices for Ensuring Originality in Written Work, "What is Plagiarism?", 2015. [Online]. Available: <http://www.plagiarism.org/plagiarism-101/what-is-plagiarism/>. [Accessed: 29- Dec- 2015].

Appendices

The appendices include further information that is not essential to be included in the main text, but nevertheless could be useful to interested readers. The following appendices should be included in the dissertation:

8.1 Appendix A - System Manual

Technical documentation is included here. These details should guide candidates who wish to continue or use your project work and allow amendments and extensions to the code. Provide program installation, compilation, and execution details. Documentation should point to locations where DLL's and reusable code can be found if they are publicly available.

8.2 Appendix B - User Manual

May include thorough and comprehensive documentation at a level, which is appropriate to the identified users. User documentation may cover all aspects of the system, with appropriate screenshots and explanations. **Failing to include such documents means that you have failed to implement a critical component of the system and it could result in not calling for project evaluation.**

8.3 Appendix C - Management Reports

In addition to producing day to day transaction reports (e.g. a payroll system should produce an individual pay sheet, coin analysis to make cash payments, EPF report, etc.) a system must produce summarised reports for the management (e.g. monthly, quarterly payments made by organisation, employees, overtime hours by employee, etc.). These reports should be included here. The usefulness of the system will be judged using these reports. **Failing to include such reports means that the candidate had failed to achieve his objectives and it could result in not calling for project evaluation.** Ensure that the reports contain meaningful information which could be obtained through your system using a sufficient amount of data.

General Guidelines

Candidates are strongly advised, while writing is in progress, to show each chapter to their supervisors for necessary feedback especially on technical content. Please follow the instructions below to minimize correction time.

- The candidates are advised to follow the typing recommendations given below to typeset their dissertations.
 - Dissertation text Times New Roman - 12pt
 - Text in tables and code listing - 11pt
 - Line spacing (preface and main text) - 1.5
 - Line spacing appendices - 1.0
 - Left margin - 37mm
 - Top/bottom/right margins - 25mm
 - Chapter heading - 24pt bold
 - Section headings - 16pt
 - Subsection headings - 14pt
 - Other headings - 12pt bold
 - Tables headings font - 11pt bold

- All pages should be numbered including Chapter 1 beginning from page 1. Use roman numerals for front matter as used in this guidelines document.

- **Plagiarism** is the presentation of another person's thoughts or words as though they were the candidate's own. The candidate should avoid this when writing his dissertation. All sentences or passages quoted in reports from other people's work have to be specially acknowledged by clear cross-referencing to author, work and page(s). Direct quotations from published or unpublished work of others should always be clearly identified as such by being placed inside quotation marks, and full reference to their source should be provided in the proper form. Equally, if another person's ideas or judgements are summarized, the candidate should refer to that person in the main text of the dissertation, and include the work referred to in the references section of the dissertation. Failure to adhere to these rules may result

in an allegation of cheating. **All suspected cheating will be reported as examination offenses.** Any illustrations, which are not the work of the candidate can be used only with the explicit permission of the originator and should be specially acknowledged. The project is an important component of the degree and plagiarism in project work is taken very seriously, and when discovered will imply severe penalties and consequences for the culprit's degree and possibly for his entire future career. Such a candidate will fail the project and the degree examination as a whole when plagiarism in project work is discovered. Candidates will not be allowed to repeat the project and other degree components for a specified number of years depending on the decision by the university. Therefore, it is important to give credit where it is due and acknowledge all work borrowed and emphasize what the candidate's distinct contribution has been in the project. **Similarity Index should be less than 20% to avoid consideration for possible plagiarism attempts.**

9.1 Writing Style

The following are some of the general points you should use them to improve writing style:

- Try to minimize mixing of unrelated issues in the same chapter. For example, in the design chapter, do not discuss testing issues. One exception to this rule would be when arguments need to justify actions related to the current topic.
- Throughout the dissertation, use standard software engineering or computer science terminology. (refer Guide to the Software Engineering Body of Knowledge Version 3.0 SWEBOK®,)
- Use a consistent naming convention. Do not refer to the same concept or “thing” by different names in different parts of the dissertation.
- Write short, focused, and coherent paragraphs. Each paragraph should have a clearly stated topic sentence that describes the main issue/context covered in the paragraph. The rest of the sentences should provide discussions on the topic sentence, or answer the question raised in the topic sentence.

- Do a grammar and style check before submission. It is ESSENTIAL that you proofread your dissertation manually before printing the final copy for submission.
- Omit needless words—redundant modifiers, pompous diction, excessive detail.
- Eliminate "narration," expressions such as "It is my opinion that," "I have concluded," "the main point supporting my view concerns," or "certainly there is little doubt as to. ". Focus attention solely on what the reader needs to know.